

Chương 1: PHÂN TÍCH ĐỘ PHỨC TẠP THUẬT TOÁN

Bài toán 1.1.1. Bài toán tìm dãy con lớn nhất

Cho dãy số $a_1, a_2, \dots, a_n$, dãy số $a_i, a_{i+1}, a_{i+2}, \dots, a_j$ với $1 \leq i \leq j \leq n$ là một dãy con của dãy số đã cho, tổng $\sum_{k=i}^j a_k$ được gọi là trọng lượng của dãy. Bài toán đặt ra tìm dãy con có trọng lượng lớn nhất.

Các cách tiếp cận giải quyết bài toán

1.1.1. Thuật toán trực tiếp

```
pair<int, vector<int>> BruteForce1(vector<int> a)
{
    int n = a.size();
    int maxSum = INT_MIN, head = -1, tail = -1;
    for (int i = 0; i < n; ++i)
    {
        for (int j = i; j < n; ++j)
        {
            int tmpSum = 0;
            for (int k = i; k <= j; ++k){
                tmpSum += a[k];
                if (tmpSum > maxSum)
                {
                    maxSum = tmpSum;
                    head = i;
                    tail = j;
                }
            }
        }
    }
    vector<int> res(a.begin()+head, a.begin()+tail+1);
    return {maxSum, res};
}
```

1.1.2. Cải tiến thuật toán trực tiếp

```
pair<int, vector<int>> BruteForce2(vector<int> a)
{
    int n = a.size();
    int maxSum = INT_MIN, head = -1, tail = -1;
    for (int i = 0; i < n; ++i)
    {
        int tmpSum = 0;
        for (int j = i; j < n; ++j)
        {
```

```

        tmpSum += a[j];
        if (tmpSum > maxSum)
        {
            maxSum = tmpSum;
            head = i;
            tail = j;
        }
    }
    vector<int> res(a.begin()+head, a.begin()+tail+1);
    return {maxSum, res};
}

```

1.1.3. Chia để trị dùng đệ quy

```

pair<int,int> MaxLeft(vector<int> a, int left, int mid)
{
    int maxSum = INT_MIN, sum = 0, left_index=-1;
    for (int i = mid; i >= left; --i){
        sum += a[i];
        if (sum > maxSum)
        {
            maxSum = sum;
            left_index = i;
        }
    }
    return {maxSum, left_index};
}

// mid in this function is mid+1 in real case.
pair<int,int> MaxRight(vector<int> a, int mid, int right) {
    int maxSum = INT_MIN, sum = 0, right_index=-1;
    for (int i = mid; i <= right; ++i){
        sum += a[i];
        if (sum > maxSum)
        {
            maxSum = sum;
            right_index = i;
        }
    }
    return {maxSum, right_index};
}

pair<int, vector<int>> MaxSubSequence(vector<int> a, int left, int
right)
{
    if (left == right) return {a[left], {a[left]}};
    int mid = left + (right-left)/2;
    pair<int, vector<int>> wL = MaxSubSequence(a, left, mid);

```

```

pair<int, vector<int>> wR = MaxSubSequence(a, mid+1, right);
pair<int, int> mL = MaxLeft(a, left, mid);
pair<int, int> mR = MaxRight(a, mid+1, right);
if (wL.first > wR.first && wL.first > mL.first+mR.first)
    return wL;
else if (wL.first < wR.first && wR.first > mL.first+mR.first)
    return wR;
else{
    vector<int> res(a.begin()+mL.second, a.begin()+mR.second+1);
    return {mL.first+mR.first, res};
}
}

```

1.1.4. Quy hoạch động

```

pair<int, vector<int>> dynamicProgramming1(vector<int> a)
{
    int n = a.size();
    vector<int> s(n);
    s[0] = a[0];
    for (int i = 1; i < n; ++i)
        s[i] = max(s[i-1]+a[i], a[i]);
    auto maxsum = max_element(s.begin(), s.end());

    /// trace
    int j = -1;
    for (int i = 0; i < n; ++i)
        if (s[i] == *maxsum)
            j = i;

    vector<int> subseq;
    for (int i = j; i > 0; --i)
    {
        subseq.push_back(a[i]);
        if (a[i] > s[i-1]+a[i])
            break;
    }
    if (a[1]<a[0]+a[1])
        subseq.push_back(a[0]);
    return {*maxsum, subseq};
}

```

Chương 2: CHIẾN LƯỢC THIẾT KẾ TRỰC TIẾP EXHAUSTED SEARCH - BRUTE FORCE

1. Một số bài toán ví dụ

Bài toán 2.1. Tính tổng sau

$$S(n) = \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

```
long long sum(int n)
{
    long long sum = 1;
    for (int i = 2; i <= n; ++i)
        sum += i*i;
    return sum;
}
```

$$T(n) = O(n)$$

```
long long sum2(int n)
{
    return (n*(n+1)*(2*n+1))/6;
}
```

$$T(n) = O(1)$$

Bài toán 2.2. Thuật toán Selection sort

```
void SelectionSort(int a[], int n)
{
    for (int i = 0; i < n-1; ++i)
    {
        int min_val = a[i], min_index = i;
        for (int j = i+1; j < n; ++j)
            if (a[j] < min_val)
            {
                min_val = a[j];
                min_index = j;
            }
        swap(a[i], a[min_index]);
    }
}
```

$$T(n) = O(n^2)$$

Bài toán 2.3. Thuật toán Sequential search

```
int SequentialSearch(int a[], int n, int key)
{
    for (int i = 0; i < n; ++i)
    {
```

```

        if (a[i] == key)
            return i;
    }
    return -1;
}

```

$$T(n) = O(n)$$

Bài toán 2.4. Bài toán so khớp chuỗi

Cho một **chuỗi văn bản** T (text) độ dài n và một **chuỗi mẫu** P (pattern) độ dài m. Tìm tất cả các vị trí trong T mà chuỗi P **xuất hiện trùng khớp**.

```

int StringMatching(string text, string pattern)
{
    int n = text.length();
    int m = pattern.length();
    for (int i = 0; i < n-m+1; ++i)
    {
        bool flag = true;
        for (int j = 0; j < m; ++j)
            if (text[i+j] != pattern[j])
                flag = false;
        if (flag)
            return i;
    }
    return -1;
}

```

$$T(n, m) = O(n \times m)$$

Bài toán 2.5. Bài toán cặp điểm gần nhau nhất

Cho một tập hợp gồm **n điểm** trên mặt phẳng hai chiều (hoặc ba chiều, nhưng thường là 2D), hãy tìm **hai điểm có khoảng cách Euclid nhỏ nhất** giữa chúng.

```

double euclideanDistance(pair<int, int> A, pair<int, int> B)
{
    return sqrt(pow(A.first-B.first, 2)+pow(A.second-B.second, 2));
}

void closetPair(vector<pair<int, int>> p)
{
    int n = p.size();
    double minDis = DBL_MAX;
    int first = 0, last = 0;
    for (int i = 0; i < n-1; ++i)
        for (int j = i+1; j < n; ++j)
        {
            double newDis = euclideanDistance(p[i], p[j]);
            if (newDis < minDis){
                minDis = newDis;
            }
        }
}

```

```

        first = i;
        last = j;
    }
}
cout << "Closest pair of points is: (" << p[first].first << ", "
<< p[first].second << ") and (" << p[last].first << ", " <<
p[last].second << ")\n";
cout << "Distance = " << minDis << "\n";
}

```

$$T(n) = O(n^2)$$

Bài toán 2.6. Bài toán người đi du lịch - TSP (Traveling Saleman Problems)

Cho một danh sách các **n thành phố** và khoảng cách giữa từng cặp thành phố, hãy tìm **chu trình ngắn nhất** đi qua **mỗi thành phố đúng một lần**, sau đó quay về thành phố xuất phát.

```

int costCalculate(int graph[15][15], vector<int> path, int n)
{
    int cost = 0;
    for (int i = 0; i < n-1; ++i)
        cost += graph[path[i]][path[i+1]];
    cost += graph[path[n-1]][path[0]];
    return cost;
}

pair<int, vector<int>> tsp(int graph[15][15], int n)
{
    vector<int> path(n), best_path(n);
    generate(path.begin(), path.end(), [dem=0] () mutable {return
dem++;});
    best_path = path;
    int cost;
    int best_cost = INT_MAX;
    do{
        cost = costCalculate(graph, path, n);
        if (cost < best_cost)
        {
            best_cost = cost;
            best_path = path;
        }
    }while(next_permutation(path.begin()+1, path.end()));

    return {best_cost, best_path};
}

```

$$T(n) = O(n!)$$

2. Bài tập

Chương 3: CHIẾN LƯỢC GIẢM ĐỀ TRỊ DECREASE AND CONQUER

1. Một số bài tập ví dụ

Bài toán 2.1. Thuật toán insertion sort

```
void InsertionSort(int a[], int n)
{
    for (int i = 1; i < n; ++i)
    {
        int key = a[i], key_index = i-1;
        while (key_index > -1 && a[key_index]>key)
        {
            a[key_index+1]=a[key_index];
            --key_index;
        }
        a[key_index+1]=key;
    }
}
```

$$T(n) = O(n^2)$$

2. Bài tập

Chương 4: CHIẾN LƯỢC CHIA ĐỂ TRI DEVIDE AND CONQUER

1. Một số bài tập ví dụ

Bài toán 4.1. Thuật toán Merge sort

```
void Merge(int a[], int left, int mid, int right)
{
    int n1 = mid-left+1;
    int n2 = right-mid;
    int L[n1+1], R[n2+1];
    for (int i = 0; i < n1; ++i)
        L[i] = a[left+i];
    for (int i = 0; i < n2; ++i)
        R[i] = a[mid+1+i];
    L[n1] = INT_MAX;
    R[n2] = INT_MAX;
    int i = 0, j = 0;
    for (int k = left; k <= right; ++k)
    {
        if (L[i] <= R[j])
        {
            a[k] = L[i];
            ++i;
        }else{
            a[k] = R[j];
            ++j;
        }
    }
}

void MergeSort(int a[], int left, int right)
{
    if (left < right)
    {
        int mid = (right+left)/2;
        MergeSort(a, left, mid);
        MergeSort(a, mid+1, right);
        Merge(a, left, mid, right);
    }
}
```

Bài toán 4.2. Thuật toán Quick sort

```
int Partition(int a[], int left, int right)
{
    int x = a[right], i = left-1;
    for (int j = left; j < right; ++j)
```



```

    {
        if (a[j] <= x)
        {
            ++i;
            swap(a[i], a[j]);
        }
    }
    swap(a[i+1], a[right]);
    return i+1;
}

void QuickSort(int a[], int left, int right)
{
    if (left < right)
    {
        int pivot = Partition(a, left, right);
        QuickSort(a, left, pivot-1);
        QuickSort(a, pivot+1, right);
    }
}

```

Bài toán 4.3. Binary Search

```

int BS1(int a[], int left, int right, int key)
{
    if (left <= right)
    {
        int mid = left + (right-left)/2;
        if (a[mid] == key)
            return mid;
        if (a[mid] < key)
            return BS1(a, mid+1, right, key);
        return BS1(a, left, mid-1, key);
    }
    return -1;
}

int BS2(int a[], int n, int key)
{
    int l = 0, r = n-1;
    while (l <= r)
    {
        int m = l + (r-l)/2;
        if (a[m] == key)
            return m;
        else if (key > a[m])
            l = m+1;
        else
            r = m-1;
    }
}

```

```
}  
    return -1;  
}
```

Bài toán 4.4.

Bài toán 4.5.

2. Bài tập

Chương 5: CHIẾN LƯỢC BIẾN ĐỔI ĐỀ TRỊ
TRANSFORM AND CONQUER

1. Một số bài tập ví dụ

Bài toán 5.1.

Bài toán 5.2.

Bài toán 5.3.

Bài toán 5.4.

Bài toán 5.5.

2. Bài tập

Chương 6: CHIẾN LƯỢC QUI HOẠCH ĐỘNG DYNAMIC PROGRAMMING

1. Một số bài tập ví dụ

Bài toán 5.1.

--

Bài toán 5.2.

--

Bài toán 5.3.

--

Bài toán 5.4.

--

Bài toán 5.5.

--

Chương 7: CHIẾN LƯỢC QUAY LUI VÀ NHÁNH CẬN BACKTRACKING – BRANCH AND BOUND

1. Một số bài tập ví dụ

Bài toán 5.1.

--

Bài toán 5.2.

--

Bài toán 5.3.

--

Bài toán 5.4.

--

Bài toán 5.5.

--